

EE675 Assignment 2: Policy Gradient for Predator-Prey Game

Shreyash

1 Problem setting

The environment is the same predator-prey game from Assignment 1, with fixed grid size $N = 4$ and discount factor $\gamma = 0.99$. A state is represented by the ordered pair

$$s_t = ((x_t^p, y_t^p), (x_t^q, y_t^q)),$$

where (x_t^p, y_t^p) is the predator position and (x_t^q, y_t^q) is the prey position. The action set is

$$\mathcal{A} = \{\text{stay}, \text{up}, \text{down}, \text{left}, \text{right}\}.$$

The predator moves first. If it catches the prey, the reward is 1 and the prey is respawned uniformly over all cells except the predator's current cell. Otherwise, the prey moves uniformly over its feasible one-step moves, including staying in place, and the catch condition is checked again. Invalid boundary moves are handled conservatively by masking them out before sampling from the neural policy.

2 Neural policy design

The policy $\pi_\theta(a \mid s)$ is represented using a PyTorch neural network implemented in `policy_network.py`. The state is encoded using normalized coordinates:

$$\phi(s) = \left[\frac{x^p - 1}{N - 1}, \frac{y^p - 1}{N - 1}, \frac{x^q - 1}{N - 1}, \frac{y^q - 1}{N - 1} \right] \in \mathbb{R}^4.$$

Thus the input dimension is 4. The output dimension is 5, one logit for each global action. The architecture used is

$$4 \rightarrow 64 \rightarrow 64 \rightarrow 5,$$

with `Tanh` nonlinearities after the two hidden layers. The final logits are converted to a categorical distribution after invalid actions are masked. This keeps the policy differentiable in the valid action probabilities while preventing probability mass from being wasted on impossible boundary moves.

3 Policy-gradient estimator

Since the original process is continuing, the implementation uses a finite truncation horizon H to estimate the discounted objective

$$J_H(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{H-1} \gamma^t R_t \right].$$

For a sampled trajectory, define the return-to-go

$$G_t = \sum_{k=t}^{H-1} \gamma^{k-t} R_k.$$

Using the likelihood-ratio trick,

$$\nabla_{\theta} J_H(\theta) = \mathbb{E} \left[\sum_{t=0}^{H-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right].$$

With M independently sampled trajectories, the Monte Carlo gradient estimate is

$$\hat{g} = \frac{1}{M} \sum_{i=1}^M \sum_{t=0}^{H-1} \gamma^t G_t^{(i)} \nabla_{\theta} \log \pi_{\theta}(A_t^{(i)} | S_t^{(i)}).$$

This is an unbiased estimator of the finite-horizon discounted policy gradient. The implementation is contained in `gradient_estimate.py`.

The PyTorch score-function computation is performed using

```
torch.distributions.Categorical.log_prob(action).
```

Then `torch.autograd.grad(objective, policy.parameters())` is used to obtain gradients of the REINFORCE surrogate objective with respect to the neural-network parameters. For Adam, the same surrogate is maximized by minimizing its negative and calling `loss.backward()` followed by `optimizer.step()`.

4 Simple stochastic gradient ascent

The simple stochastic gradient ascent update is

$$\theta_{k+1} = \theta_k + \eta \hat{g}_k.$$

This is implemented in `simple_sga.py`. For the reported run, the learning rate was chosen as

$$\eta_{SGA} = 0.02.$$

The run used 100 iterations, $M = 4$ trajectories per update, horizon $H = 60$, and $\gamma = 0.99$.

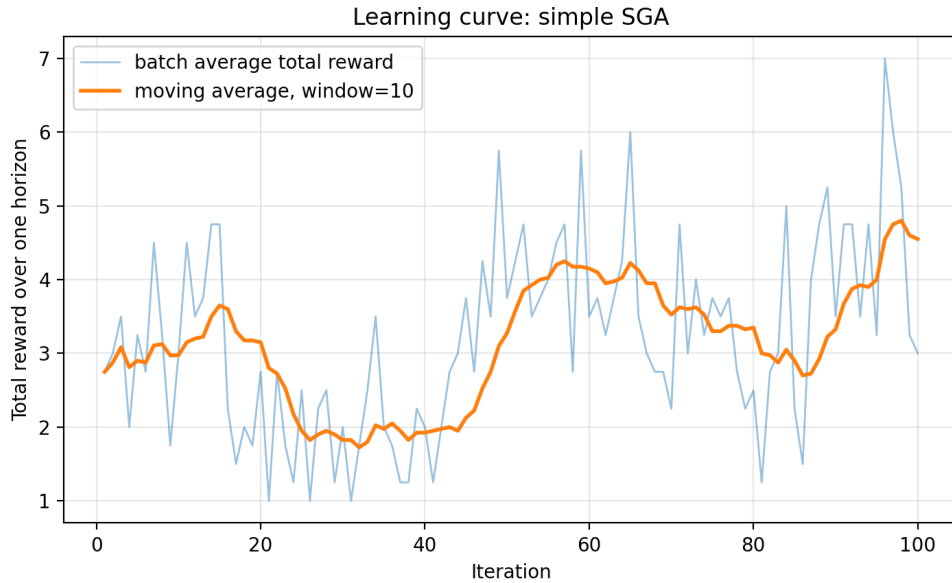


Figure 1: Learning curve for simple stochastic gradient ascent. The raw curve is noisy because the reward is sparse and each update uses a small Monte Carlo batch.

5 Adam optimizer

The Adam optimizer was also tested using PyTorch’s built-in `torch.optim.Adam`. The same policy-gradient surrogate objective was used, but parameter updates were handled by Adam’s adaptive first- and second-moment estimates. The learning rate was chosen as

$$\eta_{\text{Adam}} = 0.001.$$

The other settings were kept the same: 100 iterations, $M = 4$, $H = 60$, and $\gamma = 0.99$.

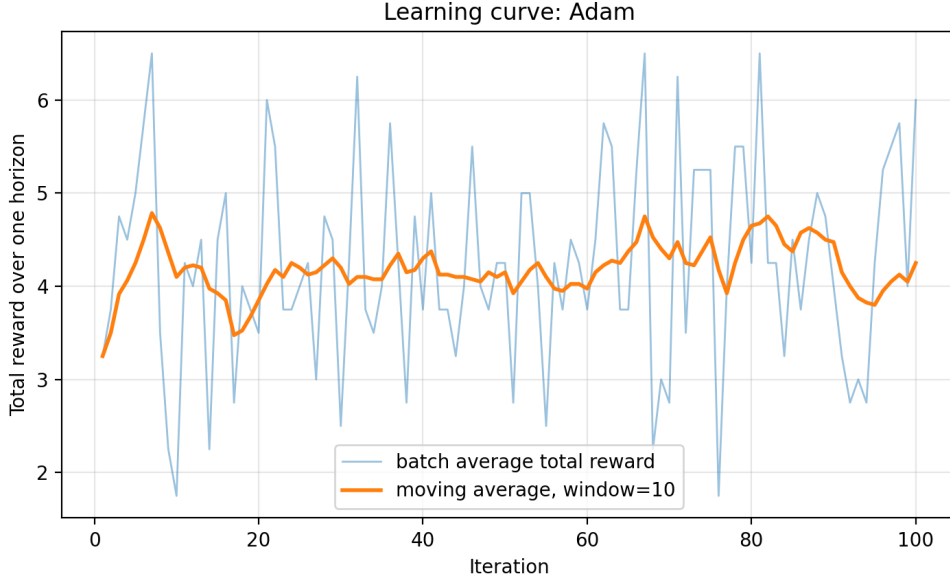


Figure 2: Learning curve for Adam. Adam is less sensitive to the raw scale of the gradient estimate, but the curve is still stochastic because it is driven by simulator rollouts.

6 Numerical summary

The table below reports simple diagnostics from `results/training_log.csv`. The plotted reward is the average undiscounted total reward collected over one horizon, averaged over the Monte Carlo trajectories in that update.

Method	First reward	Last reward	Last-10 avg.	Max reward	Last-10 discounted avg.
simple SGA	2.75	3.00	4.55	7.00	3.448
Adam	3.25	6.00	4.25	6.50	3.131

Table 1: Learning-curve summary for the generated run. Because the estimator is Monte Carlo and rewards are sparse, the correct reading is the trend of the smoothed curve rather than any single iteration.

7 File naming conventions

The code is split into separate files according to functionality:

- `environment.py`: simulator and grid/action utilities.
- `policy_network.py`: PyTorch neural policy and state encoder.

- `gradient_estimate.py`: REINFORCE gradient estimator using simulator rollouts.
- `simple_sga.py`: manual stochastic gradient ascent update.
- `train.py`: training loop for simple SGA and Adam, CSV logging, and plot generation.

8 How to reproduce

Install the dependencies and run

```
pip install -r requirements.txt
python train.py --iterations 100 --episodes 4 --horizon 60 \
    --sga-lr 0.02 --adam-lr 0.001
```

This regenerates `results/training_log.csv`, `plots/simple_sga_learning_curve.png`, and `plots/adam_learning_curve.png`.